Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Classical Knapsack Cryptosystem and its variations

Bachelor Thesis

2025
David Vavřinek

Comenius University in Bratislava
Faculty of Mathematics, Physics and Informatics

Classical Knapsack Cryptosystem and its variations

Bachelor Thesis

Study Programme:	Applied Computer Science
Field of Study:	Computer Science
Department:	Department of Applied Informatics
Supervisor:	doc. RNDr. Tatiana Jajcayová, PhD.

Bratislava, 2025
David Vavřinek

19931079

Univerzita Komenského v Bratislave
Fakulta matematiky, fyziky a informatiky

ZADANIE ZÁVEREČNEJ PRÁCE

Meno a priezvisko študenta: David Vavřinek
Študijný program: aplikovaná informatika (Jednoodborové štúdium, bakalársky I. st., denná forma)
Študijný odbor: informatika
Typ záverečnej práce: bakalárska
Jazyk záverečnej práce: anglický
Sekundárny jazyk: slovenský

Názov: Classical Knapsack Cryptosystem and its variations
Klasický Knapsack Cryptosystém a jeho variácie

Anotácia: Knapsack kryptosystém je kryptosystém s verejným kľúčom, ktorého bezpečnosť je založená na ťažkom probléme z teórie čísel - probléme Knapsack.

Cieľ: Cieľom bakalárskej práce je pochopiť, naštudovať, analyzovať a naprogramovať kryptosystém Kanpsack. Následne zhromaždiť a porovnať rôzne variácie a vylepšenia klasického kryptosystému Knapsack.

Vedúci: doc. RNDr. Tatiana Jajcayová, PhD.
Katedra: FMFI.KAI - Katedra aplikovanej informatiky
Vedúci katedry: doc. RNDr. Tatiana Jajcayová, PhD.

Dátum zadania: 20.09.2025

Dátum schválenia: 22.09.2025

doc. RNDr. Damas Gruska, PhD.
garant študijného programu

.....
študent

.....
vedúci práce

19931079

Comenius University Bratislava
Faculty of Mathematics, Physics and Informatics

THESIS ASSIGNMENT

Name and Surname:	David Vavřinek
Study programme:	Applied Computer Science (Single degree study, bachelor I. deg., full time form)
Field of Study:	Computer Science
Type of Thesis:	Bachelor´s thesis
Language of Thesis:	English
Secondary language:	Slovak

Title: Classical Knapsack Cryptosystem and its variations

Annotation: Knapsack cryptosystem is a public key cryptosystem with its security based on the difficult number theoretic problem - Knapsack Problem.

Aim: The aim of the theses is to understand, study, analyze and program the Kanpsack cryptosystem. Then collect and compare different variations and improvements of the classical Knapsack cryptosystem.

Supervisor:	doc. RNDr. Tatiana Jajcayová, PhD.
Department:	FMFI.KAI - Department of Applied Informatics
Head of department:	doc. RNDr. Tatiana Jajcayová, PhD.
Assigned:	20.09.2025
Approved:	22.09.2025

doc. RNDr. Damas Gruska, PhD.
Guarantor of Study Programme

..... Student

..... Supervisor

Acknowledgments: Tu môžete poďakovať školiteľovi, prípadne ďalším osobám, ktoré vám s prácou nejako pomohli, poradili, poskytli dáta a podobne.

Abstrakt

Slovenský abstrakt v rozsahu 100-500 slov, jeden odstavec. Abstrakt stručne sumarizuje výsledky práce. Mal by byť pochopiteľný pre bežného informatika. Nemal by teda využívať skratky, termíny alebo označenie zavedené v práci, okrem tých, ktoré sú všeobecne známe.

Kľúčové slová: jedno, druhé, tretie (prípadne štvrté, piate)

Abstract

Abstract in the English language (translation of the abstract in the Slovak language).

Keywords: knapsack, cryptography, algorithms

Contents

Introduction **1**

1 Theoretical Background **3**

1.1 Basics of Cryptography 3

1.2 P, NP, and Computational Hardness 4

1.3 Symmetric and Asymmetric Cryptography 6

1.4 The Knapsack Problem 6

1.5 Knapsack Cryptosystem 8

1.5.1 Key Generation 9

1.5.2 Encryption 9

1.5.3 Decryption 9

2 Implementation and Methodology **11**

2.1 Choice of Programming Language 11

2.2 Algorithm Design 11

2.3 Testing and Verification 11

3 Variations and Cryptanalysis of Knapsack Cryptosystems **13**

3.1 Weaknesses of the Classical Merkle–Hellman Scheme 13

3.1.1 Structural Weakness of the Trapdoor Construction 14

3.1.2 Shamir's Polynomial-Time Attack 14

3.1.3 Low-Density Attacks and Lattice Methods 14

3.1.4 Why NP-Completeness Does Not Guarantee Security Here . . . 14

3.2 Selected Variations of Knapsack Cryptosystems 14

3.2.1 Iterated and Modified Merkle–Hellman Variants 14

3.2.2 Permutation and Modular Transformation Enhancements 14

3.2.3 Chor–Rivest Knapsack-Type Cryptosystem 14

3.2.4 Other Historical Proposals (Brief Overview) 14

3.3 Comparison of Variants 14

3.3.1 Security Perspective 14

3.3.2 Efficiency and Implementation Complexity 14

3.3.3 Summary Table of Schemes and Attacks 14
3.4 Discussion 14
3.4.1 Lessons from the Rise and Fall of Knapsack Cryptography . . . 14
3.4.2 Implications for Modern Public-Key Design 14

4 Results **15**

5 Conclusion **17**

List of Figures

List of Tables

Introduction

Cryptography is a fundamental part of modern information security. It enables confidential communication by transforming data into forms that are unaccessible to unauthorized parties. In today's digital world, cryptography is is being used in almost every secure communication system, from online banking and data storage to digital signatures and blockchain technologies. Its roots go back thousands of years. From simple substitution ciphers like Ceasar cipher used in ancient civilizations, through more complex systems like the Vigenère cipher, to the formal mathematical foundations developed in the twentieth century.

Throughout it's history cryptography reliad mostly on symmetric cryptosystems, where both the sender and reciever use the same private key for both encryption and decryption. While these systems, such as DES and AES, are efficient and still widely used today, they require both parties to securely share the secret key in advance. Which comes with it's own problems

A major milestone in the history of cryptography came with the invention of public-key cryptography in the 1970s. Unlike classical symmetric systems, where both parties share the same secret key, public-key systems use a pair of mathematically linked keys. Public key for encryption and a private key for decryption. This enables secure communication between parties without exchanging keys before communication. Among the most well-known public-key systems are the RSA cryptosystem, based on the difficulty of integer factorization, and the Diffie–Hellman key exchange, which relies on the discrete logarithm problem.

Alongside the first systems such as RSA, one of the earliest public-key systems was the Knapsack cryptosystem proposed by Merkle and Hellman in 1978, which relied on the computational difficulty of the Knapsack problem, a classic example of an NP-complete combinatorial problem. The Knapsack problem asks whether a given sum can be formed by selecting specific elements from a set of numbers, and while it is easy to verify a solution, finding it is computationally hard. This asymmetry between easy verification and hard computation makes the problem appealing for cryptographic use.

Although the original Merkle–Hellman system was later shown to be insecure against certain attacks, it remains an important milestone in cryptographic research. Its concept has inspired numerous modifications and improved versions that attempt to over-

come the weaknesses of the original design. Studying these variations provides valuable insight into both the design principles of secure systems and the evolution of cryptographic thinking.

The aim of this thesis is to study, analyze, compare, and implement various versions of the Knapsack cryptosystem. The work begins with an overview of basic cryptographic concepts and the theoretical background of the Knapsack problem. It then describes the classical Merkle–Hellman Knapsack cryptosystem in detail, including its key generation, encryption, and decryption procedures. The following sections analyze its security, explain known cryptanalytic attacks, and present selected improvements and modern adaptations. Finally, an implementation of the system is provided, demonstrating its functionality and allowing comparison between different variants.

Chapter 1

Theoretical Background

1.1 Basics of Cryptography

Cryptography is a part of computer science and mathematics that focuses on protecting contents of transmitted data through insecure channels like the Internet, from unauthorized parties. Its name is derived from the greek word "kryptos" meaning hidden and "graphein" meaning writing so we could literally translate it as hidden writing. In today's world it is possible to hide any type of digital communication from text to video.

The need for secret communication came much earlier. One of the first implementations of cryptography was found in the use of non-standard hieroglyphs carved into the wall of a tomb from the Old Kingdom of Egypt [3]. This is an example of a classical cipher. Classical ciphers came before the invention of computers and for that reason they operate on letters rather than bits. In ancient times or even during the First World War, cryptography had to be performed manually using only pen and paper [1]. The most famous classical cipher is Caesar cipher. It is named after Julius Caesar, who used it during his rule of Roman Empire to protect his military communications. It works by shifting each letter in message by a number of positions in alphabet. The number of the shifted letters must be agreed upon in advance between communicating parties and can be considered as key. Even though this worked well in times of Roman Empire, it's very easy to break. There are only 26 letters in English alphabet, hence only 26 keys possible. An attacker can simply try all possible shifts to quickly recover the plaintext. This weakness later motivated the development of more secure systems.

Another popular classical cipher is the Vigenère cipher which came around 1500 years after the invention of Caesar cipher. The Vigenère cipher became popular and was later used during the American Civil War by Confederate forces and during the First World War by the Swiss Army, among others [1]. It works similarly to the Caesar cipher, but rather than shifting every letter by a constant number of positions it shifts

by values defined by key. In this context, the key is a word in which each letter represents numerical value derived from their position in alphabet. During encryption, the key is repeated until it matches the length of the message. Then, each letter of the message is shifted forward in the alphabet by the value corresponding to the matching letter of the key.

Despite the improvements, it's still quite easy to break with frequency analysis techniques which exploit the uneven distribution of letters in languages. Classical ciphers are insecure because they rely on basic operations that can be done manually and modern computers can break them with basic programs analysing letter frequencies or even brute force testing all possible keys.

As the need for secure communications grew, more sophisticated systems were developed, including mechanical rotor machines such as the Enigma and Lorenz ciphers used during the Second World War. These devices implemented rapidly changing substitution alphabets and, at the time, were considered as secure.

Their eventual breaking showed a primary flaw of classical cryptography. Security of classical and mechanical ciphers depended on complicated substitutions but they lacked computational hardness.

During the 1970s and 1980s cryptography progressed into a modern era. New theory based on well-defined mathematical principles appeared, which led researchers to realize that long-term security could not rely on secrecy or complicated mechanical design, but instead on problems that are fundamentally hard to solve. Modern cryptography is therefore built upon mathematically defined functions that are easy to compute in one direction but infeasible to reverse without secret information, called a key. These ideas led to the development of both symmetric and asymmetric cryptosystems. The next section introduces the computational principles underlying these constructions.

1.2 P, NP, and Computational Hardness

In cryptography, the security of many systems depends on problems that are believed to be computationally hard to solve. To describe meaning to computational difficulty, problems are organized into complexity classes. The groups are based on how the running time of an algorithm grows with the size of the input. One important class is $\mathcal{P}$, which contains decision problems that can be solved in polynomial time. A decision problem meaning one with a yes or a no answer is in $\mathcal{P}$ if there exists an algorithm that solves any instance in time bounded by a polynomial in the input size. Intuitively, problems in $\mathcal{P}$ are considered efficiently solvable.

Another fundamental class is $\mathcal{NP}$, which stands for nondeterministic polynomial time. A decision problem belongs to $\mathcal{NP}$ if a proposed solution, often called a cer-

tificate, can be verified in polynomial time. This means that even if we do not know how to find a solution quickly, we can check whether a given solution is correct in a reasonable time.

A major open problem in theoretical computer science is whether $\mathcal{P} = \mathcal{NP}$. Many researchers believe that $\mathcal{P} \neq \mathcal{NP}$, but this has not been proven. If it turned out that $\mathcal{P} = \mathcal{NP}$, then every problem whose solution can be verified quickly could also be solved quickly. This would have enormous consequences: many problems currently considered difficult would become easy, and most modern cryptographic systems would no longer be secure. Because no polynomial-time algorithm is known for many $\mathcal{NP}$ problems, the prevailing assumption is that $\mathcal{P} \neq \mathcal{NP}$.

Within the class $\mathcal{NP}$ there exists a subset of problems that are, in a precise sense, the most computationally difficult problems in the class. These are referred to as $\mathcal{NP}$-complete problems. A decision problem C is said to be $\mathcal{NP}$-complete if it satisfies two requirements: it belongs to $\mathcal{NP}$, and every problem in $\mathcal{NP}$ can be transformed into C by a polynomial-time reduction. If a polynomial-time algorithm were discovered for any $\mathcal{NP}$-complete problem, then all problems in $\mathcal{NP}$ would become solvable in polynomial time. Since no such algorithm is known, $\mathcal{NP}$-complete problems are generally regarded as computationally unsolvable.

One well-known $\mathcal{NP}$-complete problem is the subset-sum problem. In the decision version of this problem, we are given a set of numbers and a target sum, and we ask whether there exists a subset of the numbers whose sum equals the target. It is easy to verify whether a chosen subset achieves the target sum, but finding the correct subset appears to require exponential time in the worst case. This contrast between easy verification and difficult search is typical for $\mathcal{NP}$-complete problems and is one of the reasons why the Knapsack problem has been used in cryptography.

There is also a broader category known as $\mathcal{NP}$-hard problems. Informally, $\mathcal{NP}$-hard means that a problem is at least as difficult as any problem in $\mathcal{NP}$. More formally, a problem H is $\mathcal{NP}$-hard if every problem in $\mathcal{NP}$ can be reduced to H in polynomial time. $\mathcal{NP}$-hard problems do not need to be decision problems, and they do not need to have efficiently verifiable solutions. If an $\mathcal{NP}$-hard problem is also in $\mathcal{NP}$, then it is $\mathcal{NP}$-complete. In other words, $\mathcal{NP}$-complete problems are exactly those that lie in the intersection $\mathcal{NP} \cap \mathcal{NP}$-hard.

Modern cryptographic algorithms rely on the assumed hardness of such problems. The security of a cryptosystem is based on the fact that no efficient algorithm is known for solving its underlying mathematical problem. For instance, the RSA cryptosystem is based on the difficulty of integer factorization, which is not known to be $\mathcal{NP}$-complete but is still considered hard. The classical Merkle–Hellman knapsack cryptosystem, on the other hand, is built directly on the $\mathcal{NP}$-complete Knapsack problem. Because solving the general knapsack problem appears to require exponential time, an attacker

without the secret key faces an extremely difficult computational task. As long as no efficient algorithms are found for these kinds of hard problems, cryptographic schemes based on them remain secure.

1.3 Symmetric and Asymmetric Cryptography

Modern cryptography is divided into two main types. Symmetric and Asymmetric also known as public-key cryptography. Aim of both is to secure communication, but they differ in how the keys are used and mathematical concepts that make them work giving each approach its own strengths and weaknesses.

Symmetric cryptography follows the same basic idea that classical ciphers were built on. In this approach, all parties involved in transaction use a single shared secret key for both encryption and decryption. They must exchange this key in a secure way before communicating, so key distribution becomes an important problem.

Because of this limitation, symmetric systems are often not practical when two parties have never communicated before or do not have a secure channel for exchanging the key. This problem motivated the development of a different approach, where the communicating parties do not need to share a secret in advance. This idea led to asymmetric, or public-key cryptography.

Asymmetric cryptography works with a pair of keys: a public key that can be shared with anyone and a private key that must remain secret. Messages encrypted with the public key can only be decrypted with the corresponding private key, which removes the need for a secure key exchange. The security of these systems depends on mathematical problems that are easy to compute in one direction but extremely difficult to reverse without the private key. ... These ideas form the basis of modern public-key cryptography. Many encryption schemes rely on specific mathematical problems that are assumed to be hard to solve without the secret key. One such problem is the subset-sum, also known as the knapsack problem, which plays a central role in the cryptosystem studied in this thesis. The following sections introduce this problem and explain how it can be used to construct a public-key encryption scheme.

1.4 The Knapsack Problem

The knapsack problem, also known as the subset–sum problem, is a fundamental combinatorial decision problem. Given a sequence of positive integers

$$W = (w_1, w_2, \ldots, w_n)$$

and a target value $s \in \mathbb{N}$, the task is to determine whether there exists a binary vector

$$x = (x_1, x_2, \ldots, x_n) \in \{0, 1\}^n$$

such that

$$\sum_{i=1}^{n} x_i w_i = s.$$

In this formulation, each variable x_i indicates whether the weight w_i is included in the subset. The problem is NP–complete: verifying a proposed solution is efficient, but no polynomial–time algorithm is known for solving arbitrary instances.

Example of a general instance

Consider the sequence

$$W = (3,\ 8,\ 12,\ 19)$$

and the target value

$$s = 27.$$

To decide whether a subset sums to 27, one must explore combinations of the weights. In this case, a valid solution exists:

$$8 + 19 = 27,$$

corresponding to the vector

$$x = (0,\ 1,\ 0,\ 1).$$

For a general instance, systematically finding such a vector may require testing an exponential number of subsets in the worst case.

Superincreasing sequences

A sequence W is called *superincreasing* if every element is strictly larger than the sum of all previous ones:

$$w_i > \sum_{j=1}^{i-1} w_j \qquad \forall i \geq 2.$$

For such sequences, the subset–sum problem is solvable by a simple greedy algorithm that starts with the largest weight and works downward.

For a general subset-sum instance with an arbitrary sequence of weights, the target value may have no solution, one solution, or multiple different solutions. More importantly, without additional structure, finding a valid subset may require exponential search in the worst case. In contrast, superincreasing sequences have strong structure: each decision in the greedy process is forced, so the solution (if it exists) is recovered efficiently and deterministically. This property is the key reason why superincreasing sequences are used as a private trapdoor in the classical Merkle–Hellman construction.

Example of a superincreasing instance

Consider the sequence

$$W = (2,\ 5,\ 11,\ 23),$$

which satisfies

$$5 > 2, \quad 11 > 2 + 5, \quad 23 > 2 + 5 + 11.$$

Let the target value be

$$s = 28.$$

The greedy procedure proceeds as follows:

1. $23 \leq s$, include it: $x_4 = 1$, new $s = 5$.
2. $11 > 5$, skip it: $x_3 = 0$.
3. $5 \leq 5$, include it: $x_2 = 1$, new $s = 0$.
4. $2 > 0$, skip it: $x_1 = 0$.

The algorithm terminates with the unique solution

$$x = (0,\ 1,\ 0,\ 1),$$

corresponding to

$$23 + 5 = 28.$$

Summary

The essential feature of the knapsack problem is the strong contrast between general instances, which appear computationally intractable, and structured instances such as superincreasing sequences, which admit efficient solutions. This distinction forms the mathematical basis for constructions discussed in later chapters.

1.5 Knapsack Cryptosystem

The Merkle–Hellman cryptosystem was one of the earliest public-key encryption schemes based on the subset-sum problem [5]. It was introduced in a model where secure communication does not require pre-shared secret keys. Its design uses a superincreasing private sequence as a trapdoor and publishes a transformed sequence that should appear computationally hard to solve without the private key. Although the idea was influential, later cryptanalysis showed that the basic construction is insecure [7]. The following description focuses on the classical Merkle–Hellman construction itself, namely key generation, encryption, and decryption.

1.5.1 Key Generation

The goal of key generation is to create two representations of the same knapsack instance: one easy for the owner and one hard-looking for everyone else [5]. The process starts by choosing a superincreasing sequence $W = (w_1, \ldots, w_n)$, which will serve as the private trapdoor. Next, the key owner chooses a modulus m larger than the sum of all private weights and a multiplier r that is invertible modulo m. These values are used to hide the private sequence by modular transformation, producing a public sequence $B = (b_1, \ldots, b_n)$, where

$$b_i \equiv r\, w_i \pmod{m}.$$

In the classical description, a permutation is also applied to further obscure the original structure. The public key is the transformed sequence (and permutation, if used), while the private key consists of the superincreasing sequence and transformation parameters [5].

1.5.2 Encryption

After key generation, the sender uses the public sequence

$$B = (b_1, b_2, \ldots, b_n)$$

to encrypt an n-bit message

$$x = (x_1, x_2, \ldots, x_n), \quad x_i \in \{0, 1\}.$$

The ciphertext is computed as

$$C = \sum_{i=1}^{n} x_i b_i.$$

This means that only the public weights corresponding to message bits equal to 1 are added. From the sender's perspective, encryption is therefore straightforward and efficient, because it only requires additions over public values [5].

1.5.3 Decryption

To decrypt the ciphertext C, the key owner uses private parameters and first computes

$$S' \equiv r^{-1} C \pmod{m},$$

where r^{-1} is the modular inverse of r modulo m. Because the public weights were constructed from the private superincreasing sequence, this step maps the problem back to an equivalent subset-sum instance over the private sequence. The resulting value

S' is then solved by a greedy procedure, starting from the largest private weight and moving downward. At each step, if the current weight does not exceed the remaining sum, the corresponding bit is set to 1 and the weight is subtracted; otherwise the bit is set to 0. If permutation is used during key generation, bit positions are restored according to the secret key order after greedy recovery [5].

Example

Consider a private superincreasing sequence

$$W = (2,\ 5,\ 11,\ 23),$$

with $m = 47$ and $r = 9$, where $\gcd(r, m) = 1$ and $m > \sum W = 41$. The public key is

$$B = (18,\ 45,\ 5,\ 19),$$

computed as $b_i \equiv rw_i \pmod{m}$. For message $x = (1, 0, 1, 0)$, encryption gives

$$C = 18 + 5 = 23.$$

For decryption, $r^{-1} \equiv 21 \pmod{47}$, so

$$S' \equiv r^{-1}C \equiv 21 \cdot 23 \equiv 13 \pmod{47}.$$

Using greedy recovery on W: $13 = 11 + 2$, so recovered bits are $x = (1, 0, 1, 0)$.

Assumptions and Limitations

Although the classical Merkle–Hellman design is efficient for encryption and decryption, it assumes that the transformation from the private superincreasing sequence to the public sequence sufficiently hides the trapdoor structure. In practice, this assumption is not strong enough for the basic construction: later cryptanalysis showed that equivalent private structure can be recovered efficiently in polynomial time [7]. Therefore, the original scheme is historically important, but not secure for real-world use, which motivates the analysis of weaknesses and attacks in the next chapter.

Chapter 2

Implementation and Methodology

2.1 Choice of Programming Language

2.2 Algorithm Design

2.3 Testing and Verification

Chapter 3

Variations and Cryptanalysis of Knapsack Cryptosystems

3.1 Weaknesses of the Classical Merkle–Hellman Scheme

The classical Merkle–Hellman cryptosystem is based on a clear trapdoor idea: an easy private superincreasing subset-sum instance is transformed into a public instance that should look hard [5]. However, in the basic construction, this hiding transformation is not strong enough to remove all useful structure. As a result, the public key does not behave like a fully random hard subset-sum instance, and this opens the door to efficient cryptanalysis.

A major result was given by Shamir, who showed a polynomial-time attack against the basic Merkle–Hellman scheme [7]. The attack demonstrates that, even without knowing the private sequence directly, an equivalent private structure can be reconstructed from public information. This breaks the intended trapdoor asymmetry and shows that the original scheme is insecure in its classical form.

Another important line of attacks is connected with low-density subset-sum instances and lattice-based methods [4, 2]. In these settings, techniques related to lattice reduction can recover message information more efficiently than exhaustive search. Although subset-sum is NP-complete in general, practical cryptanalytic success depends on instance structure and parameters, not only on worst-case complexity.

Because of this, the historical lesson from classical Merkle–Hellman is that NP-completeness alone does not guarantee cryptographic security. A public-key design must ensure that the published instances do not leak exploitable structure, even if the underlying problem is hard in general [6]. These weaknesses motivated many modified knapsack proposals, discussed in the next section

3.1.1 Structural Weakness of the Trapdoor Construction

3.1.2 Shamir's Polynomial-Time Attack

3.1.3 Low-Density Attacks and Lattice Methods

3.1.4 Why NP-Completeness Does Not Guarantee Security Here

3.2 Selected Variations of Knapsack Cryptosystems

3.2.1 Iterated and Modified Merkle–Hellman Variants

3.2.2 Permutation and Modular Transformation Enhancements

3.2.3 Chor–Rivest Knapsack-Type Cryptosystem

3.2.4 Other Historical Proposals (Brief Overview)

3.3 Comparison of Variants

3.3.1 Security Perspective

3.3.2 Efficiency and Implementation Complexity

3.3.3 Summary Table of Schemes and Attacks

3.4 Discussion

3.4.1 Lessons from the Rise and Fall of Knapsack Cryptography

3.4.2 Implications for Modern Public-Key Design

Chapter 4

Results

Chapter 5

Conclusion

Bibliography

[1] Jean-Philippe Aumasson. *Serious Cryptography: A Practical Introduction to Modern Encryption*. No Starch Press, 2017.

[2] Erik F. Brickell. A survey of Low-density Knapsack cryptosystems. In *Advances in Cryptology – CRYPTO '82*, pages 3–25. Plenum Press, 1983.

[3] IBM. History of cryptography, 2024. Accessed: 2025-11-12.

[4] Jeffrey C. Lagarias and Andrew M. Odlyzko. Solving low-density subset sum problems. *Journal of the Association for Computing Machinery*, 32(1):229–246, 1985.

[5] Ralph C. Merkle and Martin E. Hellman. Hiding information and signatures in trapdoor knapsacks. *IEEE Transactions on Information Theory*, 24(5):525–530, 1978.

[6] A. M. Odlyzko. The rise and fall of knapsack cryptosystems. In Carl Pomerance, editor, *Cryptology and Computational Number Theory*, volume 42 of *Proceedings of Symposia in Applied Mathematics*, pages 75–88. American Mathematical Society, 1990.

[7] Adi Shamir. A polynomial-time algorithm for breaking the basic merkle-hellman cryptosystem. In *Proceedings of the 23rd IEEE Symposium on Foundations of Computer Science*, pages 145–152. IEEE, 1982.